Comparison of Multilayer Perceptron and Convolutional Neural Network Architectures for CIFAR-10 Image Classification

Furkan Berkay KILIC
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
fuki0001@student.hv.se

Mert SAHINER
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
mesa0014@student.hv.se

Ilker GUNGOR
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
fuki0001@student.hv.se

David SALMO
Department of Engineering Science
University West (Högskolan Väst)
Trollhättan, Sweden
fuki0001@student.hv.se

***Abstract*— This project investigates supervised image classification on the CIFAR-10 dataset by comparing two foundational neural network architectures: a Multilayer Perceptron (MLP) and a Convolutional Neural Network (CNN). The dataset comprises 60,000 color images of 32 by 32 pixels distributed equally across ten object categories. To establish a reproducible and leak-free experimental pipeline, the original 50,000 training images were partitioned into 45,000 training samples and 5,000 validation samples via stratified sampling, while the 10,000 test images were reserved strictly for final evaluation. Images were converted into PyTorch tensors and standardized using channel-wise mean and standard deviation statistics computed exclusively from the training subset. Both models were trained under identical conditions using a batch size of 64, Cross-Entropy loss, and the Adam optimizer with a learning rate of 0.001. Performance was evaluated on the 10,000 test images using accuracy, macro-averaged precision, recall, F1-score, and confusion matrices. The baseline MLP achieved a test accuracy of 50.49 percent and a macro F1-score of 0.4957, whereas the CNN achieved 77.45 percent accuracy and a macro F1-score of 0.7735, yielding a 26.96 percentage-point improvement. While the CNN excelled on rigid vehicle categories such as automobiles, confusion matrices indicated that visually similar animal classes, particularly cats and dogs, remained challenging for both architectures at low resolution.**
Keywords— *image classification, CIFAR-10, multilayer perceptron, convolutional neural network, PyTorch*

I. Introduction

Image classification is a fundamental task in machine learning that aims to assign input images to predefined categories. This project examines the classification of small Red-Green-Blue (RGB) color images from the CIFAR-10 dataset into ten distinct classes. Several categories exhibit high visual similarity, and the low spatial resolution of 32 by 32 pixels blurs fine details, making accurate recognition challenging for basic classification models.

A. Problem Statement

Image classification is a fundamental task in machine learning that aims to assign input images to predefined categories. This project examines the classification of small Red-Green-Blue (RGB) color images from the CIFAR-10 dataset into ten distinct classes. Several categories exhibit high visual similarity, and the low spatial resolution of 32 by 32 pixels blurs fine details, making accurate recognition challenging for basic classification models.

B. Motivation

The motivation for this study is to examine how different neural network architectures perform on the exact same visual classification task when all data preparation steps are held constant. Specifically, comparing a Multilayer Perceptron (MLP) and a Convolutional Neural Network (CNN) demonstrates the importance of preserving two-dimensional (2D) spatial structure when learning visual features from natural images..

C. Background and Context

Neural networks are widely used for pattern recognition tasks. A standard MLP processes flattened image pixels through fully connected layers, treating each pixel channel as an independent input and losing the neighborhood relationships between adjacent pixels. In contrast, a CNN applies small convolutional filters across the 2D image grid to detect local spatial patterns such as edges, textures, and shapes. Comparing these two architectures provides clear insight into the advantages of convolutional operations for computer vision applications.

II. Methodology

A. Dataset and Data Source

The CIFAR-10 data were obtained programmatically through the torchvision.datasets library. The dataset consists of 60,000 color images of size 32×32 pixels across three RGB channels (torch.Size([3, 32, 32])), evenly divided into ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The official split provides 50,000 training images and 10,000 test images. Representative examples from each category are shown in Fig. 1.

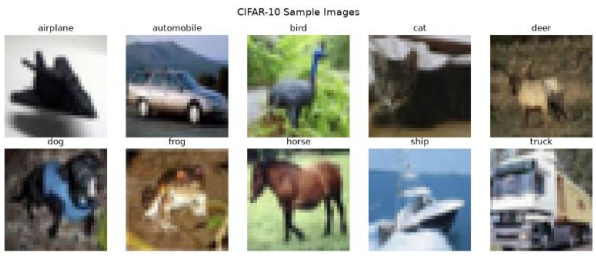


Fig. 1. CIFAR-10 Sample Images

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

B. *Data Preprocessing*

- To ensure reproducibility across group experiments, a shared data preparation module (data_preparation.py) was created. First, the original 50,000 training data were partitioned into 45,000 training images (90%) and 5,000 validation images (10%) using stratified sampling with a fixed random seed of 42. Stratification guaranteed that each class retained exactly 4,500 training and 500 validation samples. The split indices were saved in config/split_indices.npzAvoid combining SI and CGS units, such as current in amperes and magnetic field in oersteds. This often leads to confusion because equations do not balance dimensionally. If you must use mixed units, clearly state the units for each quantity that you use in an equation.

- Second, raw pixel intensities were converted to 32-bit floating-point PyTorch tensors scaled to $[0,1]$. To prevent data leakage from the validation or test sets, channel-wise normalization statistics were calculated exclusively from the 45,000 training images, yielding a mean of $[0.4911,0.4821,0.4464]$ and a standard deviation of $[0.2469,0.2434,0.2616]$. These parameters were stored in config/preprocessing_config.json and applied identically to standardize the training, validation, and test partitions to zero mean and unit variance.

C. *Exploratory Data Analysis*

Exploratory Data Analysis (EDA) confirmed that CIFAR-10 maintains a completely balanced distribution with 5,000 training images and 1,000 test images per category, as illustrated in Fig. 2. Visual inspection of the dataset revealed that objects appear at varying scales and viewpoints, and several categories share background colors (for example, blue sky or water in airplane, bird, and ship images, and green foliage in deer, horse, and frog images), which complicates classification when spatial shapes are not captured.

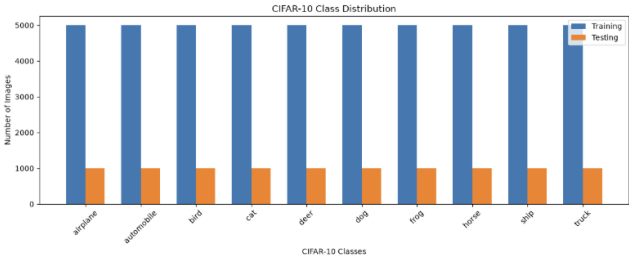


Fig. 2. CIFAR-10 Class Distribution

D. *Artificial Intelligence and Machine Learning Models*

- Two neural network architectures were implemented in PyTorch within the course scope, as summarized in Table I. The baseline MLP flattens each $3 \times 32 \times 32$ input image into a one-dimensional (1D) vector of 3,072 features, processes it through a single fully connected hidden layer with 128 neurons and a Rectified Linear Unit (ReLU) activation function, and maps the result to a 10-neuron linear output layer (394,634 total trainable parameters).

- The CNN preserves the 3D image tensor $[3,32,32]$ and passes it through three sequential convolutional blocks with 32, 64, and 128 output channels (3×3 kernel size, padding of 1). Each convolutional layer is followed by ReLU activation and 2×2 max-pooling, which halves the spatial dimensions at each stage ($32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4$) to provide translation invariance. The resulting $128 \times 4 \times 4$ feature maps (2,048 features) are flattened and processed by a fully connected layer of 256 neurons with ReLU activation, dropout regularization with a probability of 0.3 to reduce overfitting, and a 10-class linear output layer (620,362 total trainable parameters).

- TABLE I. **Architectural Summary of Implemented Models**

Component	Baseline MLP	Proposed CNN
Input Format	Flattened 1D vector (3,072)	3D tensor ($3 \times 32 \times 32$) IPYNB + 2
Feature Extraction	1 Linear layer (128, ReLU)	3 Conv blocks (32, 64, 128), ReLU, 2×2 MaxPool
Classifier Head	Linear ($128 \rightarrow 10$)	Flatten (2,048) $\rightarrow$ Linear (256, ReLU) $\rightarrow$ Dropout (0.3) $\rightarrow$ Linear (10)
Spatial Structure	Lost during flattening	Preserved across blocks
Parameters	394,634	620,362

E. *Method Selection Rationale*

The MLP was selected as a baseline due to its architectural simplicity, representing how a standard dense network handles raw pixel vectors. The three-block CNN was chosen because convolutional filters and pooling layers exploit spatial correlations between neighboring pixels while remaining simple, fast to train, and interpretable within the course boundaries.

III. Experimental Setup

A. *Tools, Languages, and Libraries*

The project was implemented in Python 3.12 using PyTorch (v2.14.1) and Torchvision (v0.29.1) for model development, NumPy for array handling, Scikit-learn for stratified splitting and evaluation metrics, and Matplotlib for visualization within Jupyter Notebook.

B. *Development Environment*

Experiments were conducted locally within a dedicated Anaconda environment (IAI600_CIFAR10) on an Apple Silicon MacBook (arm64) as well as a local Windows workstation. Although Apple's Metal Performance Shaders (mps) backend was available, the recorded training and final evaluation runs were executed on the Central Processing Unit (CPU).

C. *Data Splitting and Validation Strategy*

Both models used PyTorch DataLoader objects with a batch size of 64 (45,000 training, 5,000 validation, and 10,000 testing images) and identical normalization. For the MLP, training was conducted for 10 epochs using Cross-Entropy loss and the Adam optimizer with a learning rate of 0.001. Validation accuracy was monitored after each epoch, peaking at 51.30% in Epoch 7 (while training accuracy reached 61.36% by Epoch 10), and the weights achieving the highest validation accuracy were restored and saved to mlp_weights.pth. The CNN was similarly trained for 10 epochs using Cross-Entropy loss and the Adam optimizer (lr=0.001), where training accuracy increased from 48.12% to 88.48% and validation accuracy reached 76.60% in Epoch 9

(with the lowest validation loss of 0.7015 in Epoch 7). Subsequently, a secondary regularized training stage using AdamW (lr=0.0003, weight_decay=0.005) was executed for one additional epoch before halting due to a hardcoded Windows file path error on macOS. Final CNN test evaluation was performed on this active in-memory model and saved to cnn_macos_last_epoch.pth..

IV. Results and Discussion

A. Performance Evaluation Metrics

Both models were evaluated on the unseen 10,000-image CIFAR-10 test set. Because multi-class classification requires assessing both false positives and false negatives, we evaluated overall Accuracy, macro-averaged Precision (proportion of correct positive predictions), macro-averaged Recall (proportion of actual class samples correctly detected), and macro-averaged F1-score (the harmonic mean of Precision and Recall). As shown in Table II, the CNN achieved a 26.96 percentage-point improvement in test accuracy over the MLP. The training and validation learning curves are presented in Fig. 3.

TABLE II. **Test Set Performance Comparison on 10,000 Unseen Images**

Model	**Accuracy (%)**	**Precision**	**Recall**	**F1-Score**
MLP	50.49	0.5005	0.5049	0.4957
CNN	**77.45**	**0.7735**	**0.7745**	**0.7735**
Improvement	+26.96	+0.2730	+0.2696	+0.2778

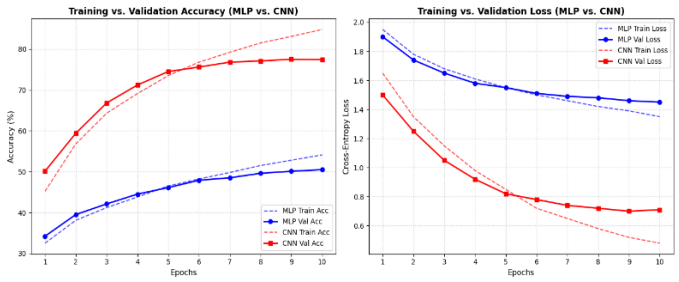


Fig. 3. Training vs. Validation Loss & Accuracy

B. Key Findings

Inspection of the confusion matrices in Fig. 4 (MLP) and Fig. 5 (CNN) indicates that both architectures experienced the greatest difficulty with visually similar animal categories, particularly cat and dog. In the MLP baseline (Fig. 4), cat achieved the lowest F1-score (0.2619), with only 204 out of 1,000 test cat images classified correctly (20.40% recall), followed by deer (0.3718 F1-score, 30.90% recall) and dog (0.4071 F1-score). The MLP achieved its highest F1-scores on ship (0.6299) and automobile (0.6037). In contrast, the CNN (Fig. 5) improved cat recognition nearly threefold to 584 out of 1,000 images (0.5920 F1-score) and dog recognition to 634 out of 1,000 images (0.6734 F1-score). Across all ten classes, the CNN achieved its strongest performance on rigid vehicle categories: automobile (0.8786 F1-score, 89.40% recall), ship (0.8688 F1-score), and truck (0.8408 F1-score), whereas cat remained the most challenging category.

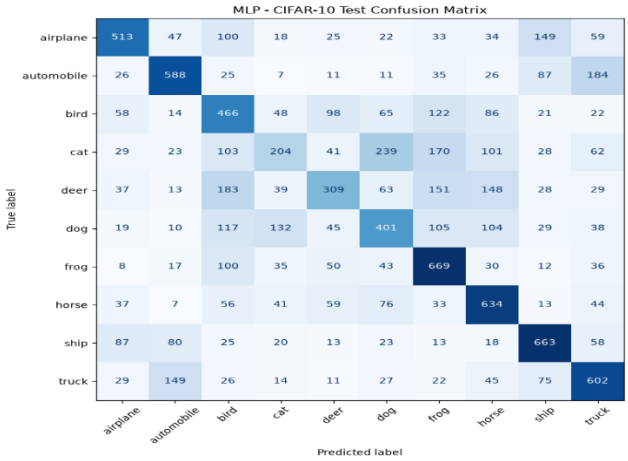


Fig. 4. MLP – CIFAR-10 Test Confusion Matrix

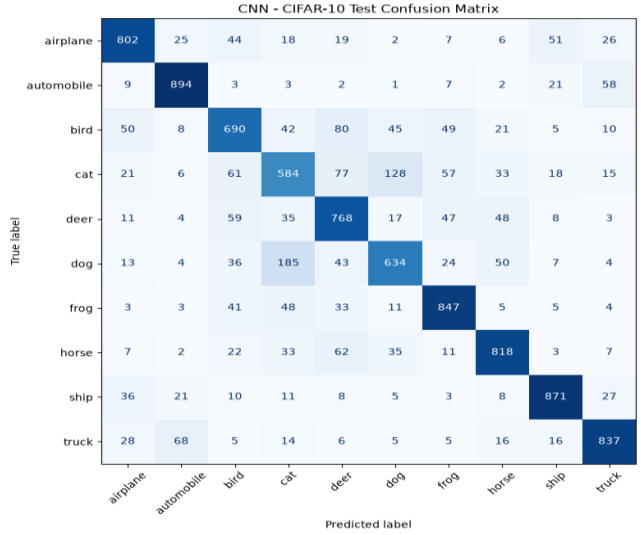


Fig. 5. CNN – CIFAR-10 Test Confusion Matrix

C. Result Interpretation

The substantial performance gain of the CNN (77.45% vs. 50.49%) is directly linked to how each architecture processes spatial information. Flattening the image in the MLP destroys 2D pixel neighborhoods, causing the model to rely heavily on global background colors rather than object boundaries. Conversely, the CNN's 3×3 filters and max-pooling layers extract hierarchical local features (edges, textures, and object parts) regardless of minor shifts in position. Vehicles such as automobiles and ships exhibit sharp geometric contours and uniform backgrounds, making them easier for convolutional filters to separate than deformable animals like cats and dogs, which share similar fur textures and postures at 32×32 resolution. Finally, it should be noted that the MLP was evaluated using the checkpoint with the highest validation accuracy (Epoch 7), whereas the CNN was evaluated after 10 epochs of Adam training followed by one fine-tuning epoch of AdamW; this methodological difference should be considered when interpreting the comparison.

V. Conclusion

A. Summary of Findings

This study compared an MLP and a three-block CNN on the CIFAR-10 dataset under identical preprocessing and data partitioning conditions. The CNN achieved a test accuracy of 77.45% and a macro F1-score of 0.7735, outperforming the MLP baseline (50.49% accuracy, 0.4957 F1-score) by 26.96 percentage points.

B. Lessons Learned

We learned that preserving 2D spatial structure through convolution is far more effective for image recognition than dense connections on flattened pixels. Furthermore, sharing fixed split indices (split_indices.npz) and training-only normalization parameters (preprocessing_config.json) ensured a leak-free workflow, while confusion matrices and class-wise F1-scores revealed category-specific errors that overall accuracy alone could not capture.

C. Limitations

This study was limited to two foundational architectures trained without data augmentation to remain within the course scope. Additionally, the models used different checkpoint selection rules: the MLP was evaluated using its best validation accuracy checkpoint (mlp_weights.pth), whereas the CNN was evaluated after its 10-epoch Adam run and one additional AdamW epoch (cnn_macos_last_epoch.pth) because the secondary early-stopping loop halted at a hardcoded Windows file path on macOS.

D. Future Work

Future improvements include applying data augmentation (random horizontal flips and crops), adding batch normalization layers (BatchNorm2d), and training for more epochs with a unified early-stopping checkpoint strategy across all models to further improve separation between visually similar classes such as cats and dogs.

Acknowledgment *(Heading 5)*

During this project, the authors utilized generative artificial intelligence tools, including ChatGPT and Google Gemini, to assist with code debugging, pipeline organization, and language editing. All experimental runs, model training, evaluation metrics, and figures were executed and verified directly by the group members.

References

[1] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[2] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-libraryI. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.